Abstract classes and Interfaces

Member Access / Visibility

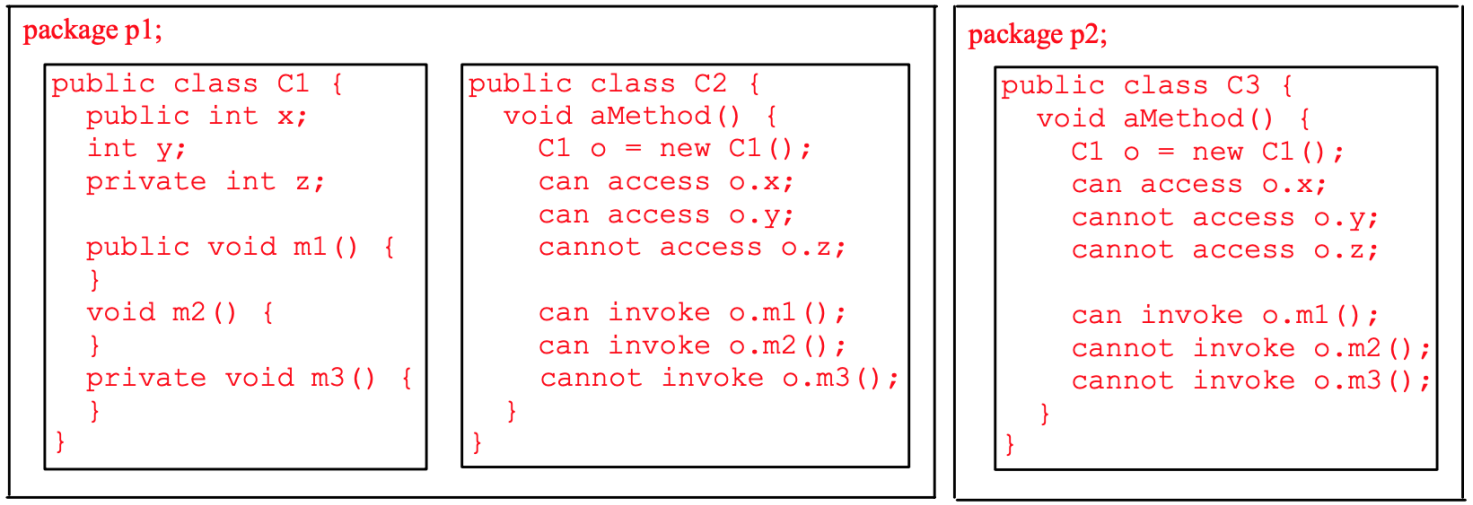
If a class has no modifier (the default, also known as package-private), it is visible only within its own package

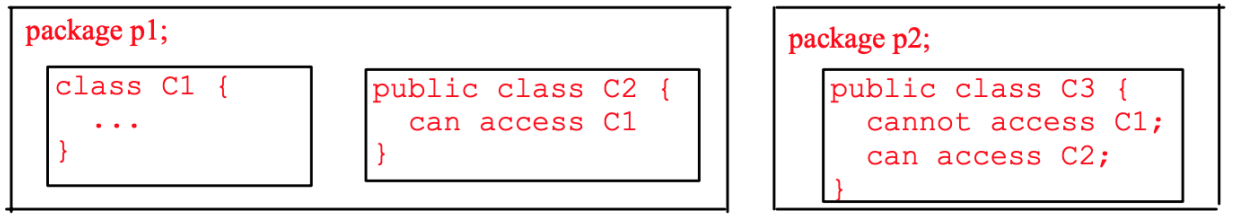
The protected modifier specifies that the member can only be accessed within its own package (as with package-private) and, in addition, by a subclass of its class in another package.

Modifier	Class	Package	Subclass	World
public	Y	Y	Y	Y
protected	Y	Y	Y	N
no modifier	Y	Y	N	N
private	Y	N	N	N

Member Access / Visibility

The `private` modifier restricts access to within a class, the default modifier restricts access to within a `package`, and the `public` modifier enables unrestricted access.





ArrayList of Objects

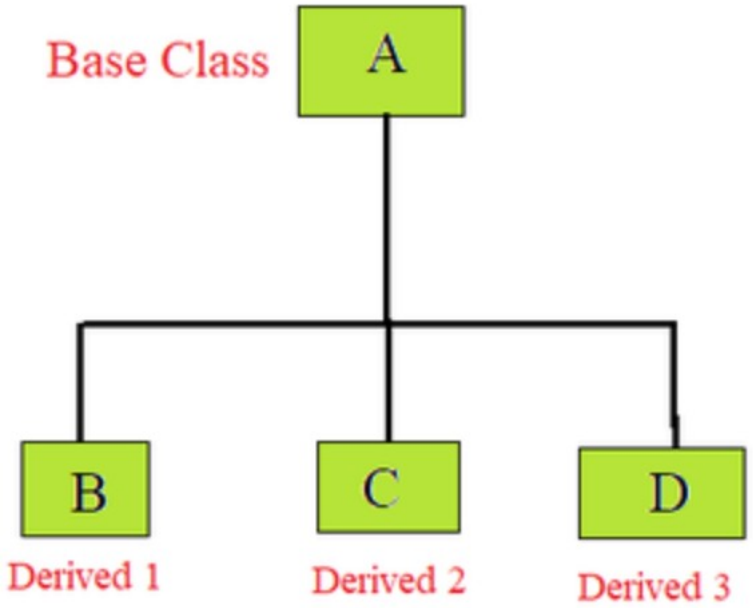
Because all classes are derived from the Object class, programmers can take advantage of runtime polymorphism in order to create a collection (e.g., ArrayList) of objects of various class types and perform operations on the elements.

```
public static void main(String[] args) {
    ArrayList<Object> objList = new ArrayList<Object>();

    // Add new instances of various classes to objList
    objList.add(new Object());
    objList.add(12);
    objList.add(3.14);
    objList.add(new String("Hello!"));
    objList.add(new Business("ACME", "5 Main St"));

    // Print list of Objects
    printArrayList(objList);
}
```

Is-a versus has-a relationship




```
public class PersonInfo {
   public String firstName;
   public String birthdate;

   ...
}

public class ChildInfo extends PersonInfo {
   public String schoolName;

   ...
}

public class MotherInfo extends PersonInfo {
   public String spousename;
   public ArrayList<ChildInfo> childrenData;
  ...
}
```

Mother is a Person
Child is a person

Is-a versus has-a relationship

```
public class ChildInfo {
   public String firstName;
   public String birthDate;
   public String schoolName;

   ...
}

public class MotherInfo {
   public String firstName;
   public String birthDate;
   public String spouseName;
   public ArrayList<ChildInfo> childrenData;

   ...
}
```

The 'has-a' relationship. A MotherInfo object 'has a' String object and 'has a' ArrayList of ChildInfo objects, but no inheritance is involved.

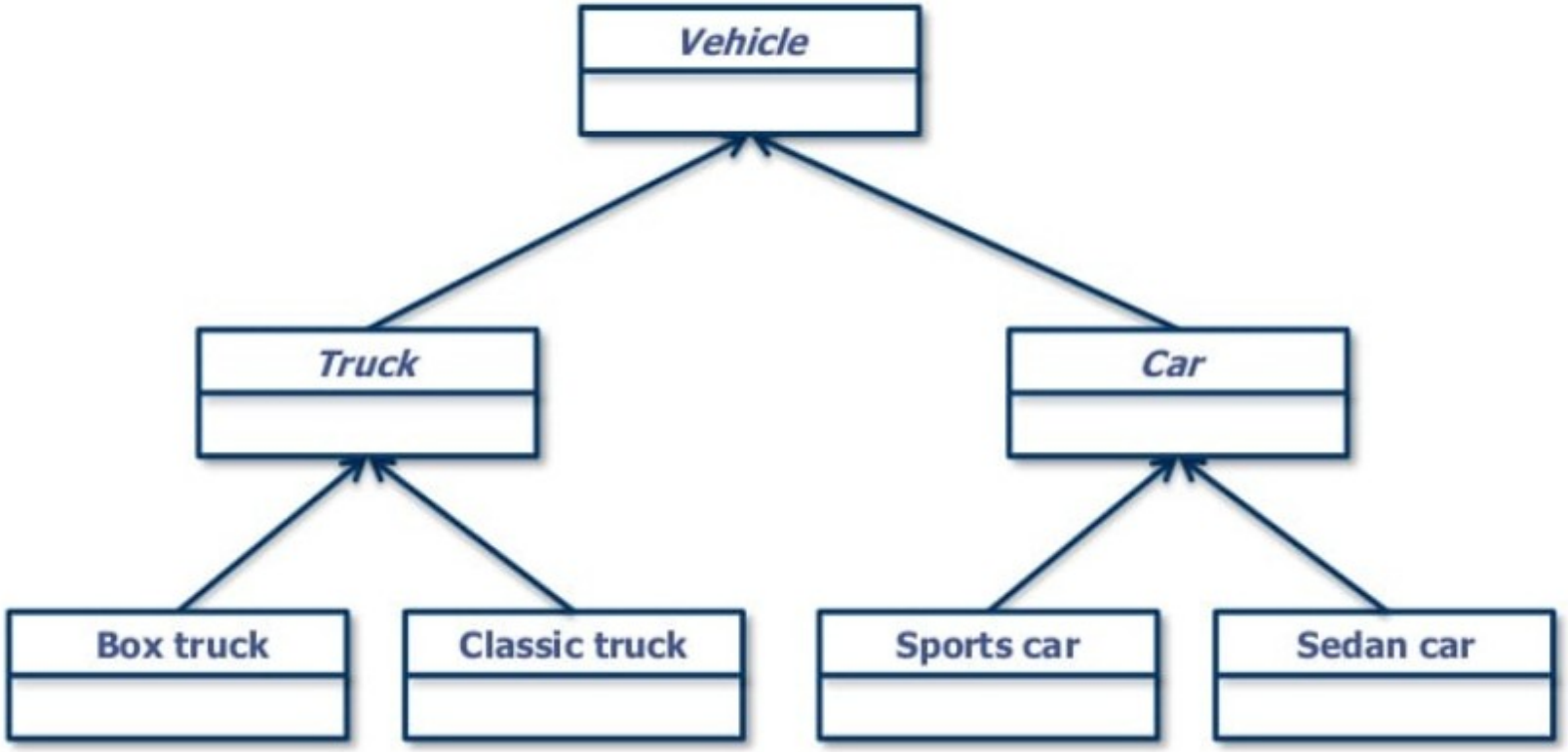
This is an example of composition. Composition should not be confused with inheritance.

Abstract classes and Interface

- Abstraction
- Abstract class
  - Formal definition
  - Example
  - When to use abstract class
- Interface
  - Formal definition
  - Example
  - When to use interface
- Difference between abstract class and interface

Abstraction

- Abstraction is a process by which concepts are derived from the usage and classification of literal ("real" or "concrete") concepts.



Abstract Classes

- An **abstract class** is a class that guides the design of subclasses but *cannot itself be instantiated as an object*. It is the superclass for a subclass and specifies how the subclass must be implemented.
  - An abstract class is denoted by the keyword **`abstract`** in front of the class definition.
- An **abstract method** is a method that is not implemented in the abstract class; thus all *derived classes **must override** the function*.
  - An abstract method is denoted by the keyword **`abstract`** in front of the method signature.
- A **method signature** defines the method's name and parameters.
  - E.g `abstract double computeArea();`
- A **concrete class** is a class that is not abstract, and hence can be instantiated.

Interfaces

- An **interface** can specify a set of abstract methods that an implementing class *must override and define*. In an interface, an abstract method does not need the `abstract` keyword in front of the method signature.
- To create an interface, you use the keyword **`interface`**.

- Any class that implements an interface must:
  - List the interface name after the keyword **`implements`** (not `extends`)!!
  - Override and implement the interface's abstract methods.

- *A class can implement multiple interfaces but cannot extend multiple classes!!*

Interfaces

- Interface methods do not have a body - the body is provided by the "implement" class
- On implementation of an interface, you must override all of its methods
- Interface methods are by default `abstract` and `public`
- Interface attributes are by default `public`, `static` and `final`
- An interface cannot contain a constructor (as it cannot be used to create objects)

Why And When To Use Interfaces?

1) To achieve security - hide certain details and only show the important details of an object (interface).

2) Java does not support "multiple inheritance" (a class can only inherit from one superclass). However, it can be achieved with interfaces, because the class can **implement** multiple interfaces. **Note:** To implement multiple interfaces, separate them with a comma (see example below).

Interfaces Summary

- Interface is a set of method signatures.
- Interface itself does not implement these methods.
  - These methods can only be implemented by classes that inherit the interface by *overriding*.
  - The implementations can have different functionalities.
- Interface cannot store instance variables.
  - All variables declared inside interface are ***implicitly*** **public static final variables (constants).**
- Interface cannot have method implementations, except for static methods.
  - All instance methods declared inside interface are ***implicitly*** **public and abstract**.

Abstract class vs Interface

- Use abstract class when:
  - The superclass and child classes are related
  - You want to define non-static and non-final fields
  - You expect the classes that extends an abstract class have many common methods and fields.
- Use interface when:
  - It's a total abstraction – all methods must be implemented by the child classes that implements this interface
  - You want your classes to implement more than one interface (multiple inheritance)
  - *You want to specify the behavior of a particular data type, but not concerned about who implements its behavior.*

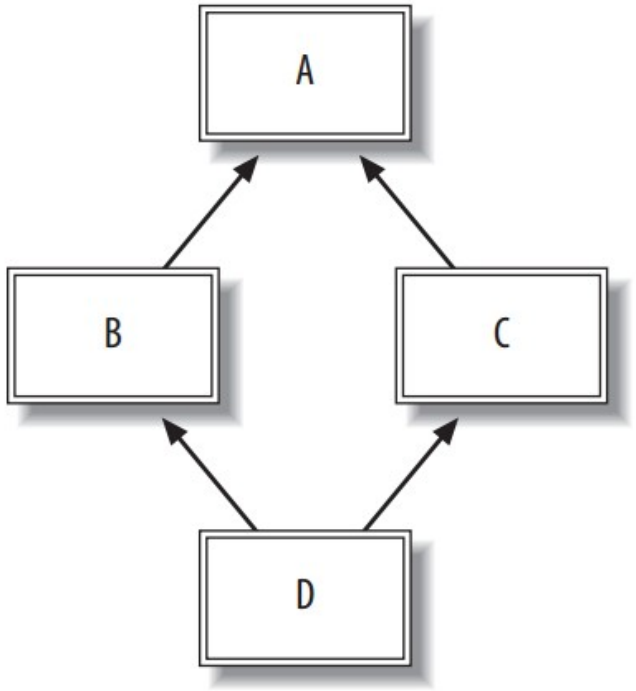
Abstract class vs Interface

- Abstract classes can do everything interfaces can do and more, but this *does not* mean that it is always better to use them than interfaces.

- One important difference between interfaces and abstract classes is that
  - a class may choose to implement arbitrarily many interfaces
  - but it can extend just one abstract class.

- That is why an interface often forms the top of an inheritance hierarchy.
- *IOP: Interface-Oriented Programming*

The Diamond Problem



- *Think about it: why multiple inheritance is prohibited in Java? What problem may be caused by multiple inheritance?*

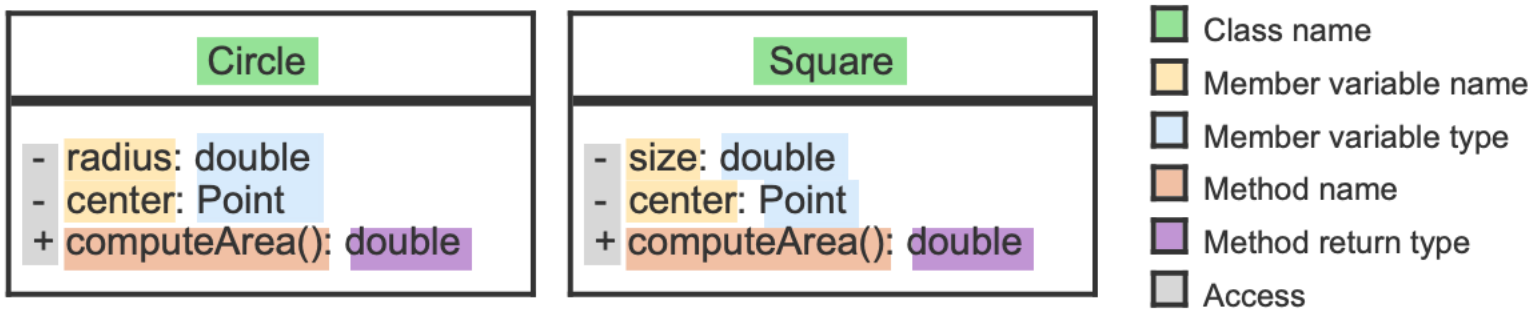
- **Diamond problem**
  - It's an ambiguity that arises when two classes B and C inherit from A, and class D inherits from both B and C.
  - If there's a method in A that B and C have overridden, and D does not override it, then which version of the method does D inherit: that of B, or that of C?

UML Diagram

The **UML** Class diagram is a graphical notation used to construct and visualize object-oriented systems. A class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's:

- classes,
- their attributes,
- operations (or methods),
- and the relationships among objects.

UML Diagram



Captions

1. One box exists for each class. The class name is centered at the top.
2. Class members are listed in the box below. Member variables have a name followed by a colon and the type.
3. Each member method's name and return type is listed similarly.
4. Private and public access is noted to the left of each member. A minus (-) indicates private and a plus (+) indicates public.

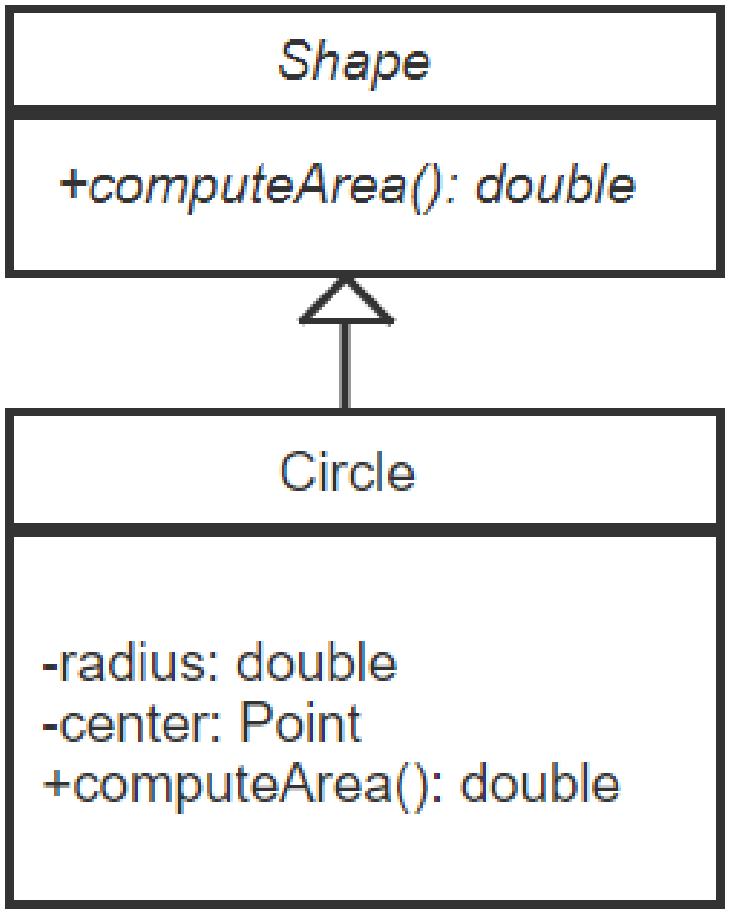
UML Diagram

A class represent a concept which encapsulates state (**attributes**) and behavior (**operations**). Each attribute has a type. Each **operation** has a **signature**



UML Diagram for inheritance

- UML uses an arrow with a solid line and an unfilled arrowhead to indicate that one class inherits from another. The arrow points toward the superclass.
- UML uses italics to denote abstract classes.



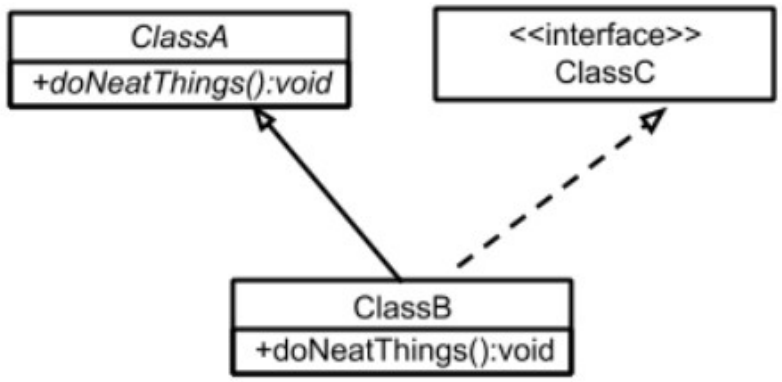
UML Diagram for interface

```
public abstract class ClassA {
   public abstract void doNeatThings();
}

public interface ClassC {

}

public class ClassB extends ClassA implements
ClassC {
   @Override
   public void doNeatThings() {
      System.out.println("Does neat things!");
   }
}
```



- UML Diagrams denote interfaces using the keyword *interface*, inside double angle brackets, above the class name. Classes that implement the interface have a dashed line with an unfilled arrow pointing at the interface.

In-Class activity

Sketch UML Diagram for inheritance chain

Class X has data members int a, b, and c.

a is accessible only within class X, whereas b is accessible to any of X's subclasses, and c is accessible to any class.

has member functions getA, setA, and multABC

Class Y extends Z and has member function multAB().

Class Z inherits directly from class X and has members int[] d and function printA().